基于 MongoDB 的智慧图书馆数字资源管理系统设计与实现

姓名	学号
康天姿	202414870106
高凯青	202414870102
马新鹭	202414870104

第 1 章 项目概述

本项目以智慧图书馆数字资源管理为应用场景，针对图书信息、读者信息、借阅记录及电子书文件的管理需求，采用 MongoDB 搭建数字资源管理平台。项目使用任务书提供的 50 条图书数据、30 条读者数据和 3000 条借阅流水，完成数据库设计、业务功能开发及分布式部署实验。

项目主要分为三个部分：一是完成MongoDB技术选型与数据模型设计，建立books、readers、borrow_logs、audit_logs 等集合；二是实现单机数据库管理和 Python 应用程序，完成信息检索、借阅统计、索引优化、用户权限管理，并利用 GridFS 实现电子书及封面图片的存储、查询和下载；三是搭建 MongoDB 副本集与分片集群，验证数据同步、故障转移和数据分布等功能。

项目成员分工如下：

康天姿：主要负责任务三、任务四及任务五至七的实施与验证，并参与任务一、任务二的方案检查和内容修改：完成单机数据库部署、数据导入、查询统计、索引优化及权限配置；开发 Python 应用，实现图书、读者、借阅等业务功能，并完成 GridFS 文件管理和操作日志；完成副本集与分片集群部署及相关实验，验证故障转移、数据分布和查询路由；完成 Compass 图形化验收、结果比对及交付材料整理。

高凯青：主要参与任务五副本集及任务六分片集群的环境搭建、配置调试与实验验证，负责相关部署资料、配置说明及实验过程记录的整理，参与三节点副本集及分片集群的初期部署和配置调试；协助项目文档汇总、图表规范化及内容一致性核查，确保成果完整、规范且符合验收要求。

马新鹭：主要负责任务一技术选型、任务二数据建模、任务四 GridFS 方案设计与模块开发、任务六分片方案论证及报告统筹工作：完成四类 NoSQL 差异对比、业务场景 CAP 分析与核心风险梳理；完成全量集合数据模型设计，明确字段选型、嵌套拆分依据与 borrow_logs 集合类型，输出索引规划；完成 borrow_logs 分片键主备选方案论证与实验设计；同时统筹报告撰写排版、验收标准对齐。

第 2 章 技术选型论证

2.1 关系型数据库与 NoSQL 数据库对比

本项目需要管理图书、读者、借阅流水和电子书文件。MySQL 能够实现这些业务，但在处理现有数据时需要进行一些额外设计。例如，部分图书缺少 tags 字段，部分读者没有 fav_categories 字段，需要通过允许空值等方式处理；图书的 location 包含楼层、书架和复本数等信息，若拆分为独立表，查询馆藏信息时还需要进行关联。随着借阅流水增加，后续还需考虑分库分表及相应的路由方案，电子书文件也需要单独确定存储方式。

相比之下，MongoDB 可以将 location 作为嵌套文档保存，也允许同一集合中的文档包含不同字段，能够减少部分数据结构转换和关联查询。对于后续的系统扩展，MongoDB 提供副本集和分片集群；电子书及封面则可以通过 GridFS 存储。因此，MongoDB 在数据结构适配和后续架构扩展方面更符合本项目的需求。两种数据库的具体差异见表 2-1。

表 2-1　两种方案在本项目中的实际差异

类型	代表	适用场景及本项目分析
键值型	Redis	低。适合缓存和按键读取。本项目需要多条件检索与聚合统计，Redis 更适合作为辅助缓存
文档型	MongoDB	高。覆盖嵌套馆藏、不齐整字段、电子书存储，同时用分片承载流水增长，单一技术栈满足全部业务
宽列型	HBase	中。写入吞吐高但交互式查询弱，只能做流水归档与离线分析，无法支撑前台业务
图数据库	Neo4j	低。本项目是借阅增删改查与统计，不存在复杂关系遍历，优势无法发挥

2.2 四类 NoSQL 选型

四类 NoSQL 数据库的数据组织方式不同，适用场景也不同。结合本项目的检索、统计和扩展需求，比较如下。

表 2-2 四类 NoSQL 适配度

类型	代表	适用场景及本项目分析
键值型	Redis	低。适合缓存和按键读取。本项目需要多条件检索与聚合统计，Redis 更适合作为辅助缓存
文档型	MongoDB	高。覆盖嵌套馆藏、不齐整字段、电子书存储，同时用分片承载流水增长，单一技术栈满足全部业务
宽列型	HBase	中。写入吞吐高但交互式查询弱，只能做流水归档与离线分析，无法支撑前台业务
图数据库	Neo4j	低。本项目是借阅增删改查与统计，不存在复杂关系遍历，优势无法发挥

2.3 CAP 理论下的业务取舍

副本集运行时，节点之间可能发生网络分区。发生分区时，系统需要在一致性和可用性之间作出取舍，分区容错则是分布式部署必须面对的条件。本项目根据操作对数据准确性的要求，采用不同处理方式，见表 2-3。

表 2-3 本项目的一致性取舍

业务场景	优先考虑	处理方式
普通馆藏查询	可用性	可采用 secondaryPreferred 读取从节点，允许短暂延迟，保证普通浏览的查询可用性
库存及借阅资格校验	一致性	使用 primary 从主节点读取业务数据；无法确认最新状态时不继续办理借阅
正式借阅流水写入	一致性	设置 writeConcern：{w：″majority″}，要求多数节点确认；无法获得多数确认时不报告操作成功
统计报表、排行榜、审计展示	可最终一致	允许秒级延迟，事后同步即可

普通馆藏浏览可以容忍副本延迟，但库存判断和正式借阅写入必须优先保证数据正确。读取偏好设为 primary 只表示从主节点读取，不能单独等同于线性一致性，还需结合读写关注进行配置。

2.4 BASE 适用边界

BASE 强调基本可用、软状态和最终一致性，适用于允许数据短暂不同步的场景。

本项目的分类统计、年度借阅趋势、排行榜和统计结果集合均由业务数据计算得到，更新稍有延迟不会影响借阅办理，可以接受最终一致性。审计日志的查询展示也可容忍副本同步延迟，但日志记录本身仍需可靠保存。

库存扣减、归还状态、正式借阅流水和读者权限会直接影响业务判断，不能仅依赖最终一致性。这些操作应在关键数据可靠写入后再反馈成功。

2.5 选型结论与风险应对

本项目选择 MongoDB 作为主数据库，主要考虑其对嵌套字段和可选字段的适配能力，以及对副本集、分片和 GridFS 的支持，能够减少数据结构调整和后续部署改造。

主要风险及应对：（1）借阅操作涉及多个集合。办理借阅时需要修改 books. stock 并新增 borrow_logs 记录，两步操作若未能同时完成，可能造成库存与流水不一致。单机部署阶段采用条件更新控制库存，再写入流水；若写入流水失败，则执行补偿并通过对账检查异常数据。副本集或分片集群环境可使用多文档事务，但需考虑额外开销。（2）分片键选择不当会造成数据分布不均。 如果分片键取值集中，部分数据和写入请求可能落在同一分片，影响扩展效果。设计时应结合字段基数、写入分布和常用查询条件比较候选方案；部署后检查各分片的数据量及 chunk 分布，发现倾斜时及时调整。

面向非技术决策者的选型说明：平台既要管理图书、读者和借阅记录，也要保存电子书及封面。选择 MongoDB，能够让这些数据按业务需要组织，并为今后的数据增长和系统扩容留出空间。库存和借阅等关键操作仍会设置相应的数据确认措施，保证业务记录可靠。

第 3 章 数据模型设计

本章完成智慧图书馆数据库逻辑结构、BSON 字段定义、索引规划，输出统一数据规范，支撑单机业务开发与后续分布式实验。

3.1 逻辑结构层次

整体采用数据库，集合，文档，字段四层结构。平台初始包含四个业务集合：books、readers、borrow_logs 和 audit_logs，分别用于馆藏管理、读者管理、借阅记录和操作审计。单机实现阶段另建 stat_result 集合，用于保存分类统计结果。电子书及封面图片通过 GridFS 存储，并使用 ebook_name 与图书数据关联。

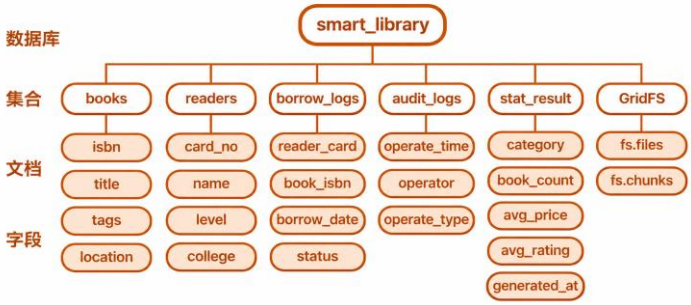


图 3-1　智慧图书馆数字资源平台数据库逻辑结构层次图

数据导入约束 允许 tags、fav_categories、ebook_name、未归还记录 return_date 字段缺失；email 可为 null。原始日期统一转为 BSON Date；不修改原始 ISBN，不为缺失字段补默认值。

3.2 集合与 BSON 字段定义

根据平台业务需求，将数据划分为馆藏图书、读者信息、借阅流水、统计结果和操作审计五个集合。各集合分别承担对应的数据存储与管理功能，具体字段及 BSON 类型如下。

books 集合存储图书基本信息、馆藏属性及电子书关联信息，是馆藏检索和库存管理的主要数据来源。

表 3-1 books 馆藏图书集合

字段	BSON 类型	业务说明
isbn	String	图书唯一 ISBN-13
title/author/publisher/category	String	图书基础信息
price/rating	Double	定价、读者评分
stock	Int32	在架库存
tags	Array<String>	图书标签，可选
publish_date	Date	出版时间
borrow_count	Int32	历史借阅次数
location	Object	嵌套文档：floor（String）、shelf（String）、copies（Int32）
ebook_name	String	关联电子书文件名，可选

约束：stock 与 location. copies 分开存储，业务以 stock 作为库存判断依据。

readers 集合存储读者基本资料、联系方式及阅读偏好，为读者查询和借阅业务提供数据支持。

表 3-2 readers 读者集合

字段	BSON 类型	业务说明
card_no	String	读者唯一借阅证号
name/college/phone/level	String	读者基础信息
email	String/Null	联系邮箱，可为 null
register_date	Date	注册时间
fav_categories	Array<String>	偏好分类，可选

borrow_logs 集合记录读者借阅和归还行为，为借阅历史查询及相关统计提供数据来源。

表 3-3 borrow_logs 借阅流水集合

字段	BSON 类型	业务说明
borrow_date	Date	借阅时间
return_date	Date	归还时间，未归还无此字段
reader_card/book_isbn	String	关联读者、图书
status	String	借阅状态
duration_days	Int32	借阅天数，归还后赋值

约束：历史数据保留原值；新记录仅归还时填写 duration_days。

表 3-4 stat_result 统计派生集合

字段	BSON 类型	业务说明
category	String	分类分组主键

字段	BSON 类型	业务说明
book_count	Int32	图书数量
avg_price/avg_rating	Double	平均值，保留 2 位小数
generated_at	Date	统计快照时间

约束：以 category 作为分类标识，通过 upsert 更新对应分类的统计结果，避免重复生成相同类别的文档。

表 3-5 audit_logs 审计日志集合

字段	BSON 类型	业务说明
operate_time	Date	操作时间
operator/operate_type/target_collection/target_id	String	操作主体、类型、操作对象
operate_detail/client_ip	String	操作描述、客户端 IP

3.3 关键选型

字段类型和集合结构根据数据特点及业务操作确定。

（1）字段类型选择。 价格和评分采用 Double 类型，便于数值比较、范围筛选和聚合计算；日期字段统一采用 BSON Date，支持时间范围查询、排序和按年月统计；库存、借阅次数等计数字段采用 Int32，符合整数计数的使用场景，避免出现无意义的小数值。

（2）馆藏位置采用嵌套文档。 floor、shelf 和 copies 与图书信息联系紧密，通常一起读取和维护，采用嵌套结构可以减少额外关联查询。当馆藏位置需要独立维护、被多个业务对象共享，或需要进行复杂的位置级查询时，可以考虑拆分为独立集合，通过位置标识建立关联。

（3）日志集合采用普通集合。borrow_logs 中的记录在图书归还时需要更新状态、归还时间和借阅时长。Capped 集合达到容量限制后会覆盖较早记录，且不允许更新操作增加文档大小，不适合本项目的借阅记录管理。audit_logs 需要长期保留操作记录，支持按时间倒序查询最近 100 条，也不适合自动覆盖历史数据。因此两者均采用普通集合。其中，borrow_logs 创建时使用默认普通集合配置，不设置 Capped 专用的 capped、size 和 max 参数，并为后续分片保留条件。

（4）统计结果单独存储。分类统计结果写入 stat_result 集合，保存图书数量、平均价格和平均评分，避免每次查询时重复执行完整聚合。

3.4 索引规划

根据馆藏检索、读者借阅记录、借阅统计和审计日志查询等业务需求规划索引。唯一索引用于保证业务标识的唯一性，复合索引结合查询条件和排序字段设计，多键索引用于支持数组字段匹配。具体索引配置及用途见表 3-6。

表 3-6 索引清单

集合	索引	索引类型	业务用途
books	{isbn:1}	唯一索引	图书主键查询更新
books	{category:1, price:1}	复合索引	分类 + 价格筛选
books	{tags:1}	多键索引	标签条件匹配
books	{publish_date:1}	单键索引	出版时间筛选
books	{rating:-1}	单键索引	评分降序 Top10

集合	索引	索引类型	业务用途
readers	{card_no:1}	唯一索引	读者主键查询
borrow_logs	{reader_card:1, borrow_date:-1}	复合索引	读者借阅记录排序查询
borrow_logs	{book_isbn:1}	单键索引	图书借阅统计
borrow_logs	{borrow_date:1}	单键索引	借阅时间趋势统计
audit_logs	{operate_time:-1}	降序索引	查询最近 100 条操作日志

本项目共规划 10 个业务索引，其中 books 集合 5 个、readers 集合 1 个、borrow_logs 集合 3 个、audit_logs 集合 1 个，不包含 MongoDB 自动创建的_id 索引。复合索引按照实际查询条件安排字段顺序，以兼顾条件筛选和结果排序。

3.5 本章核心设计决策汇总

本章结合业务数据特点与后续扩展需求，形成六项主要设计决策：

（1）价格、评分采用 Double，日期采用 BSON Date，计数字段采用 Int32，使字段类型与实际用途相匹配；

（2）保留原始数据中的字段差异，对可选字段允许缺失，email 允许为 null；

（3）馆藏位置采用嵌套文档，减少关联查询，并在位置需要独立管理时考虑拆分；

（4）借阅流水和审计日志均采用普通集合，满足状态更新、历史记录保留及后续分片扩展需求；

（5）索引围绕业务查询场景设计，兼顾查询效率与维护开销；（6）分类统计结果单独存入 stat_result 集合，书名模糊检索采用正则表达式实现。

第 4 章 功能实现

本章基于单机 MongoDB 实例完成智慧图书馆平台核心业务开发，完成数据初始化、馆藏检索、借阅统计、索引优化、权限管控、Python 应用封装，验证文档模型在业务场景下的实际能力。

4.1 单机环境与数据初始化

实验部署于 CentOS Stream 10，使用独立 MongoDB 8.0.32 实例，端口 27018，独立数据目录与配置文件，和系统默认实例隔离。开发端同时使用 MongoDB Compass 可视化工具与 PyMongo 驱动，兼顾图形化调试与业务代码开发。

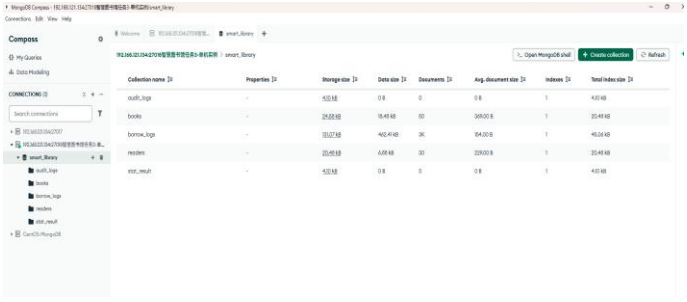


图 4-1 初始化集合文档数量

业务数据库命名 smart_library，导入任务给定数据集：50 条图书、30 条读者、3000 条借阅流水。导入阶段严格遵循第 3 章数据模型规范，所有日期字段转换为 BSON Date；允许部分字段缺失，不补默认值。导入完成后核对集合文档总数、关键字段类型，校验原始数据没有发生篡改、丢失，为后续检索、统计实验提供干净的基

线数据集。

环境隔离的意义：独立实例避免和其他实验互相干扰，账号、索引、业务集合完全独立，方便复现实验现象。

4.2 检索与统计功能

4.2.1 馆藏检索

基于 MongoDB 原生查询语法实现 6 类馆藏检索，覆盖范围筛选、数组标签匹配、模糊匹配、日期范围、字段存在性判断、排序分页，对应系统前台各类图书查询业务。

表 4-1　核心检索功能汇总

编号	功能描述	核心算子	结果条数
Q1	计算机类且价格 50~100 元	gte/lte	8
Q2	同时包含“编程”“入门”标签	$all	1
Q3	书名包含“Python”	$regex	2
Q4	出版年份 2021~2023	Date 范围查询	5
Q5	缺少 tags 字段	$exists: false	4
Q6	评分降序 Top10	sort/limit	10



图 4-2　六类馆藏检索执行输出

每一类查询模拟真实业务使用场景：Q1 用于分类价格筛选；Q2 实现多标签同时匹配；Q3 实现书名模糊搜索；Q4 筛选指定出版时段图书；Q5 用于筛选标签缺失的脏数据；Q6 实现高分图书榜单。全部查询执行结果与业务预期条数一致。

4.2.2 统计分析

利用 MongoDB 聚合管道 aggregate 实现 5 项统计业务，完成分组、表关联、排序，把分类统计结果写入派生集合 stat_result，避免重复执行消耗算力。

表 4-2　核心统计功能汇总

编号	功能描述	核心算子	结果规模
S1	分类藏书量、均价与均分	$group/$project/$sort	6 类
S2	学院借阅总量 Top5	$lookup/$unwind/$group/$sort/$limit	5 条
S3	年月借阅趋势	$dateToString/$group/$sort	36 个月
S4	热门图书 Top 10 及馆藏位置关联	$group/$sort/$limit/$lookup/$unwind/$project	10 本
S5	分类统计结果落库	$set/upsert	6 条

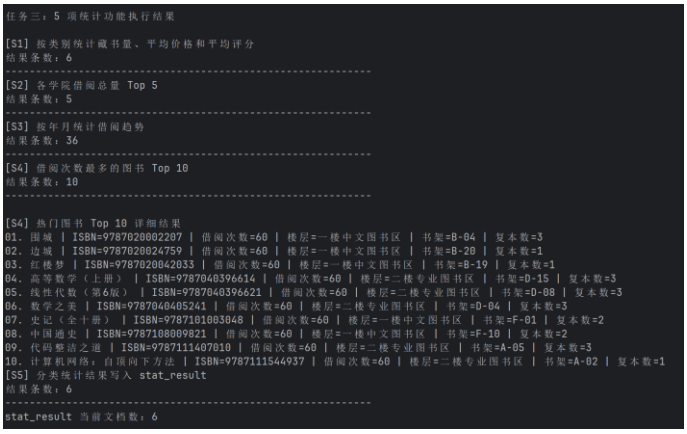


图 4-3　五项统计功能执行结果及热门图书馆藏位置关联验证

S1 按图书类别统计藏书量、平均价格和平均评分；S2 通过 $lookup 关联读者集合，统计各学院借阅总量并取前 5 名；S3 对借阅时间进行格式化分组，统计 36 个月的借阅趋势；S4 按图书 ISBN 汇总借阅次数，排序后选取前 10 本热门图书，再通过 $lookup 关联 books 集合，获取对应的书名及馆藏位置（location），输出 ISBN、书名、借阅次数和馆藏信息；S5 将分类统计结果通过 upsert 持久化至 stat_result 集合，便于后续直接读取统计结果，减少重复聚合计算。

4.3 索引优化与性能验证

依据第 3 章的索引规划，单机环境共创建 10 个业务索引，涵盖唯一索引、复合索引、多键索引和时间排序索引。为验证索引的优化效果，选取图书类别与价格组合查询、读者借阅历史查询两组典型场景，对比建立索引前后的执行计划及查询耗时。

首先，通过 explain("executionStats")检查查询执行情况。类别与价格组合查询在无索引时采用全表扫描（COLLSCAN），扫描 50 条文档；建立 ix_books_category_price 复合索引后转为索引扫描（IXSCAN），扫描文档减少至 8 条。读者借阅历史查询在无索引时需要扫描全部 3000 条借阅流水，并进行内存排序；建立 ix_borrow_reader_date 复合索引后，仅扫描 20 条匹配记录，且不再需要独立的内存排序阶段。执行计划对比见图 4-4。考虑到数据规模较小，执行计划记录的查询耗时均显示为 0ms，无法直接体现实际耗时差异，因此进一步采用重复计时方式进行测试。每组测试均先预热 100 次，再连续执行 2000 次正式查询，独立重复 5 轮，以平均单次耗时作为比较指标。测试结果如下：

表 4-3　两组查询索引建立前后平均耗时对比

查询场景	建立索引前	建立索引后	平均耗时变化
图书类别与价格查询	0.9883ms	0.9475ms	下降约 4.1%
读者借阅历史查询	2.3050ms	1.5946ms	下降约 30.8%

从测试结果看，两组查询的索引优化效果存在差异。图书查询的平均耗时仅小幅下降，且单轮测试存在波动，说明在 50 条文档的小规模集合中，索引带来的实际耗时改善有限。相比之下，借阅历史查询在五轮测试中均表现出耗时下降，平均耗时降低约 30.8%，与扫描文档数量减少及内存排序消除的执行计划结果一致。重复计时结果见图 4-5。

综合来看，索引能够减少不必要的文档扫描，并改善包含筛选和排序条件的查询效率，其优化效果与数据规模和查询方式有关。因此，本项目根据实际业务查询需求建立索引，在提高检索效率的同时，避免因索引冗余增加额外的写入与维护开销。



图 4-4 索引执行计划 executionStats 对比



图 4-5 两组查询五轮重复计时结果

4.4 用户权限与访问控制

实例开启 security. authorization 访问控制，遵循最小权限原则创建三套业务账号，划分普通查询、业务读写、数据库管理三类角色，业务程序不直接使用超级账号。

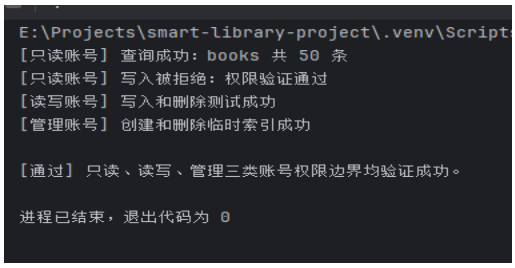


图 4-6 三类账号权限边界验证

其中 LibReader 为只读账号，分配 read 角色，仅支持集合查询，写入、建索引操作会被拒绝，适配报表查询类业务；LibWriter 为读写账号，分配 readWrite 角色，可完成集合增删改查，供 Python 主程序调用，支撑图书、读者、借阅流水日常业务；LibAdmin 为管理账号，分配 dbOwner 角色，仅用于建索引、创建用户等数据库运维工作，不参与日常业务调用。经过功能验证，只读账号执行插入操作返回权限拒绝，读写账号可正常完成业务 CRUD，管理账号能够执行索引与用户管理，各账号权限边界清晰，实现了业务访问隔离。

4.5 应用程序封装

基于 PyMongo 搭建模块化 Python 程序，拆分为 7 个低耦合模块：数据库连接模块、馆藏管理、读者管理、借阅管理、统计模块、数字资源模块、审计日志模块。数据库连接统一封装，账号信息通过配置读取，不在业务代码硬编码。



图 4-7（a） Python 主菜单

图 4-7（b）书名模糊检索示例

程序主菜单一共提供 10 项业务功能，完整覆盖：馆藏增删改查、读者信息维护、借阅归还业务、统计报表、数字资源管理、审计日志查询。

项目设计审计日志机制：所有新增、删除、导出等高危操作，程序自动向 audit_logs 集合写入操作记录，记录操作人、操作类型、操作对象、时间，实现业务行为可追溯。

同时程序做到拓扑解耦：业务逻辑与数据库部署形态解耦。未来切换副本集或者分片集群，只需要修改数据库连接配置，不需要大规模修改业务代码，为第 6 章分布式环境对接做好准备。

第 5 章 GridFS 数字资源管理

本章复用单机 MongoDB 实例，基于 GridFS 实现电子书 PDF、封面图片的文件存储，完成元数据设计、上传下载功能开发与验证。

5.1 GridFS 方案与存储结构

GridFS 为 MongoDB 内置的大文件存储方案，可突破 BSON 文档 16MB 的大小限制，依靠两套内置集合协同完成文件存储：fs.files 保存文件名、文件大小、上传时间以及自定义业务元数据；fs.chunks 存放二进制分块数据，通过 files_id 与 fs.files 建立关联。选用 GridFS 能够将电子书、封面图片与馆藏业务数据统一存放在同一数据库，复用现有权限与连接体系，无需额外部署第三方存储组件。本次实验所用测试文件均小于默认分块大小，因此每个文件仅生成单个数据块。

5.2 元数据与业务关联

在 fs.files.metadata 增加自定义元数据，用来和 books 图书记录做关联：isbn 关联图书编号、file_type 区分 ebook 电子书/cover 封面、original_filename 保存原始文件名。

实验共上传 3 份电子书、3 张封面，依靠 ISBN 实现双向业务查询：首先书名检索，然后查询 books 拿到 ISBN，再通过 metadata 查找 GridFS 资源；输入 ISBN 直接查询对应电子书与封面；从 GridFS 元数据取出 ISBN 反向查询图书信息。

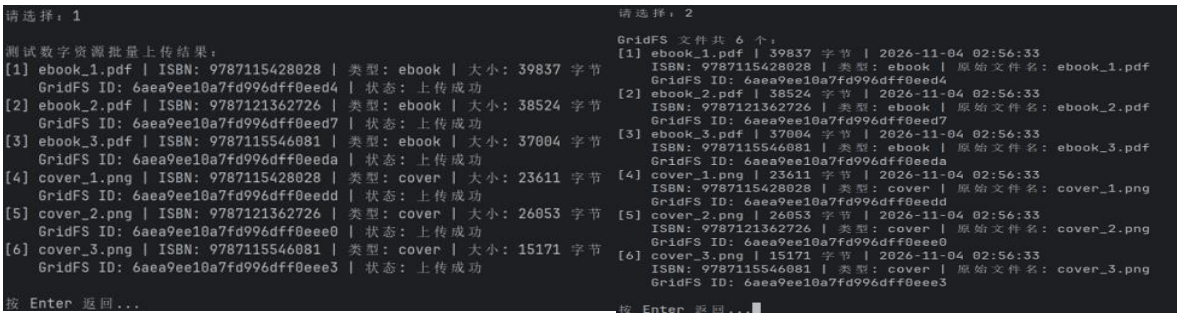
5.3 功能模块实现

开发 gridfs_manager.py 模块，封装 GridFS 全部业务接口，复用项目统一数据库连接与审计日志组件，对外提供：文件上传、资源列表、按 ISBN 查询、按书名定位

电子书、文件下载、字节一致性校验。 所有上传、下载操作自动写入 audit_logs 审计集合；连接信息统一配置，切换副本集 / 分片集群无需修改模块业务代码。

5.4 实验验证

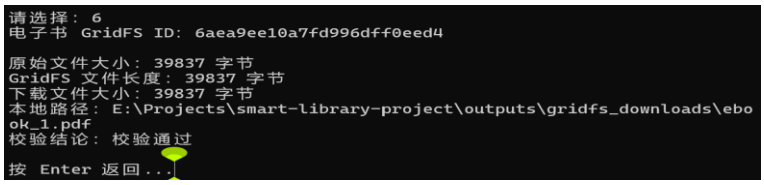
使用 LibWriter 读写账号，通过主程序的“数字资源管理”子菜单，批量上传 3 本电子书 PDF 和 3 张封面图片，共计 6 份测试资源。上传、文件清单查询及按书名检索的过程如图 5-1 所示。



（a）批量上传 3 个 PDF 和 3 张封面　　（b）GridFS 六个文件清单



（c）按书名查询电子书



（d）GridFS 文件下载及字节一致性校验

图 5-1 GridFS 文件上传、清单查询与按书名检索验证

其中，（a）为批量上传结果，6 个文件均上传成功；（b）为 GridFS 文件清单，能够查看文件名称、关联 ISBN、文件类型、字节大小及 GridFS 文件 ID；（c）为按书名关键词检索电子书的结果，程序能够正确定位对应资源；（d）为文件下载及字节一致性校验结果，选取 ebook_1.pdf 进行验证，原始文件大小、GridFS 文件长度和下载文件大小均为 39837 字节，程序校验通过。

上传完成后，对存储记录和文件完整性进行核对。fs.files 集合共生成 6 条文件记录，文件元数据完整；fs.chunks 集合共生成 6 个分块，每个文件对应 1 个分块。进一步选取文件进行下载校验，源文件、GridFS 中存储的内容与下载文件的字节数据完全一致，验证了文件存储与下载过程的完整性。测试资源及上传结果汇总见表 5-1。

表 5-1 GridFS 测试资源汇总

文件名	关联 ISBN	文件类型	文件字节大小	上传时间
ebook_1.pdf	9787115428028	ebook	39837	2026-11-04 02:56:33.856
ebook_2.pdf	9787121362726	ebook	38524	2026-11-04 02:56:33.868
ebook_3.pdf	9787115546081	ebook	37004	2026-11-04 02:56:33.872
cover_1.png	9787115428028	cover	23611	2026-11-04 02:56:33.877

文件名	关联 ISBN	文件类型	文件字节大小	上传时间
cover_2. png	9787121362726	cover	26053	2026-11-04 02:56:33. 882
cover_3. png	9787115546081	cover	15171	2026-11-04 02:56:33. 886

GridFS 模块已集成至主程序的“数字资源管理”子菜单，支持资源上传、列表查询、按 ISBN 和书名检索以及文件下载，并将上传、下载操作写入 audit_logs 集合。实验结果表明，该模块能够完成电子书及封面资源的存储、检索与下载，文件元数据关联正确，下载文件与源文件保持字节一致。

第 6 章 架构演进

本章在单机部署的基础上，完成副本集高可用与分片集群的部署及实验验证。副本集与分片集群基础设施由成员 B 负责部署，成员 C 负责分片键论证及借阅流水分片实验，在独立分片集群中完成 `borrow_logs` 集合 3000 条数据的导入、分片分布检查及查询路由验证。实验环境运行于 CentOS 虚拟机，MongoDB 版本为 8. 0. 32，副本集与分片集群均启用 keyfile 内部认证与访问控制。

6. 1 演进目标与拓扑

单机部署将数据和服务集中在单个实例上，节点故障可能导致服务中断，存储与处理能力也受到单机资源限制。针对上述问题，系统采用两条演进路径：

（1）副本集： 通过多个数据节点保存数据副本，在主节点发生故障时通过选举完成主节点切换，提高系统的可用性与故障恢复能力。

（2）分片集群： 根据分片键将数据分布到多个分片，分散单节点的数据存储与读写压力，为系统容量和处理能力的水平扩展提供支持。

表 6-1 实验组件与端口规划

组件	副本集名称	端口
数据节点 node1/node2/node3	libraryRS	27117 / 27118 / 27119
Config Server	configRS	27219
Shard 1	shard1RS	27211
Shard 2	shard2RS	27212
mongos 路由	—	27200

libraryRS 中 node1 的优先级设置为 2，node2、node3 均设置为 1，使 node1 在满足选举条件时具有更高的主节点选举优先级，为验证原主节点恢复后的选举行为提供依据。分片集群按照 Config Server → 两个 Shard → mongos 的顺序启动，停止时按相反顺序执行。

6. 2 副本集部署与高可用验证

在保留任务三、任务四单机数据库作为业务基线的前提下，本项目使用独立端口、独立数据目录和独立配置文件部署三节点 libraryRS 副本集。副本集验证使用 replica_test 数据库中的 replication_probe 测试集合，重点检查数据复制、读取偏好、主节点故障转移、认证及成员管理等功能。books、readers 和 borrow_logs 等业务集合仍保留在原单机数据库中，本次实验不涉及业务数据的整体迁移，实验结论主要针对副本集基础设施及其高可用机制。

6. 2. 1 部署边界与拓扑

表 6-2　副本集实验环境与节点配置

项目	实际配置
实验环境	CentOS Stream 10；MongoDB 8.0.32
副本集	libraryRS
正式节点	192.168.121.134:27117、27118、27119
临时节点	192.168.121.134:27120，用于成员增删演练
原单机基线	27018，保留不变，不原地改造成副本集

三个副本集节点运行在同一台 CentOS Stream 10 虚拟机中，每个实例使用独立的数据目录、日志目录和配置文件。该部署方式适用于课程实验中的进程级故障转移验证，但不能等同于跨服务器、跨主机或跨故障域的生产级高可用。

6.2.2 三节点初始化与数据同步

三个 MongoDB 实例启动后初始化 libraryRS，副本集状态形成 1 个 PRIMARY 和 2 个 SECONDARY，三个成员 health 均为 1。主节点向 replica_test.replication_probe 写入测试文档后，分别在 27118 和 27119 上读取到相同记录，说明复制链路和从节点读取配置均正常。

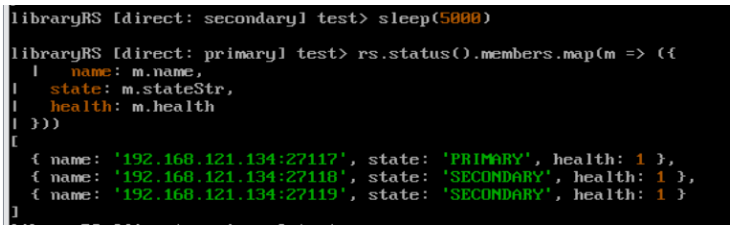


图 6-1　libraryRS 三节点副本集主从角色状态

图 6-1　直接显示了正式三节点的成员地址、角色和健康状态，证明副本集已形成稳定的三节点拓扑。主节点写入及两个从节点读取的过程证据见附录 A。

6.2.3 读取偏好验证

本项目实际验证了 primary 和 secondaryPreferred 两种读取偏好。primary 将读取固定到 PRIMARY，适合借阅状态、库存和权限等要求较新数据的业务；secondaryPreferred 优先读取 SECONDARY，在没有可用 SECONDARY 时回退到 PRIMARY，适合馆藏浏览和统计分析等能够容忍短暂复制延迟的场景。两种读取偏好的验证截图见附录 A。

表 6-3　副本集读取偏好及适用业务场景

读取偏好	实测含义	适用业务
primary	从 PRIMARY 读取最新可写状态	借阅状态、库存、权限校验
secondaryPreferred	优先从 SECONDARY 读取，不可用时回退 PRIMARY	馆藏浏览、统计分析

6.2.4 故障转移与客户端恢复

正式故障演练前，Windows 端 Python/PyMongo 连续读写探针已通过包含三个成员和 replicaSet=libraryRS 的连接串完成正常状态测试。随后停止故障前的 PRIMARY 192.168.121.134:27117。副本集将 27117 标记为不可达，27118 自动升

为 PRIMARY，27119 保持 SECONDARY；Python 客户端通过拓扑重新发现继续完成读取和写入。

表 6-4　故障转移期间客户端探针实测结果

事件	实测时间或结果
故障前最后一次识别 27117 的成功写入	21:51:26.096
第一次识别 27118 的成功读取	21:51:27.122
客户端识别新 PRIMARY 的时间差	约 1.026 s
切换后第一次读取耗时	25.6 ms
探针错误记录	未检索到显式 READ ERROR 或 WRITE ERROR

本次实验中探针没有记录显式读写错误，但日志显示客户端约在 1.026 s 后识别到新的 PRIMARY，不能将该结果表述为“完全无中断”。该时间是客户端拓扑重新发现与恢复读写的观测结果；mongod --shutdown 从开始执行到进程完全退出的时间不作为副本集选举耗时。

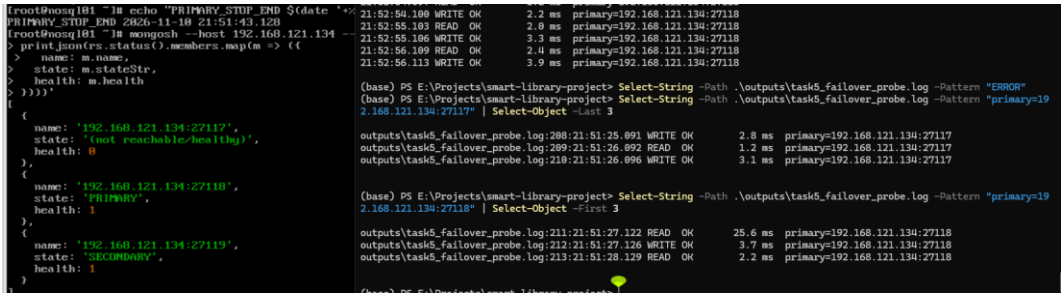


图 6-2　PRIMARY 故障后的副本集重新选举与客户端自动恢复

故障恢复后重新启动原 27117 节点，该节点以 SECONDARY 身份重新加入副本集，三节点恢复为健康状态。由此形成“27117 为 PRIMARY → 27117 停止 → 27118 升为 PRIMARY → 客户端继续读写 → 27117 以 SECONDARY 重新加入”的完整过程链。

6.2.5 访问控制与成员管理

副本集启用 authorization，并使用共享 libraryRS.key 实现节点间内部认证。实验中创建 libraryAdmin 管理账号；未认证执行 listDatabases 被返回“Command listDatabases requires authentication”，管理员认证后 connectionStatus 显示 libraryAdmin@admin 和 root@admin，说明未认证访问被拒绝且管理员认证成功。



（a）未认证访问被拒绝



（b）管理员认证成功并正常访问两个 Shard

图 6-3 副本集访问控制验证

成员管理实验中，临时节点 192.168.121.134:27120 成功加入并成为 SECONDARY，四个成员 health 均为 1；随后移除该节点，成员列表恢复为三个正式节点，最终状态为 1 PRIMARY + 2 SECONDARY，三个节点均 health=1。完整加入、移除和认证配置过程见附录 A。

6.2.6 验收结论与实验边界

任务五达到任务书规定的验收结论：三节点正常，主从角色明确；PRIMARY 停止后完成自动故障转移并有时间与角色变化记录；副本集认证已启用，未认证访问被拒绝且管理员认证成功。与此同时，本实验的三个节点位于同一台虚拟机，结论限定为进程级高可用和复制机制验证，不延伸为跨主机生产级高可用。

6.3 分片集群部署与扩展验证

任务五通过副本集解决了节点级故障转移问题，任务六进一步面向借阅流水持续增长后的存储与写入扩展需求，验证 MongoDB 分片集群的路由和数据分布能力。本实验不重新设计业务集合，而是继续使用任务三、四确定的 smart_library 数据库和 borrow_logs 集合，在独立的分片集群实例上完成部署与验证。任务五的三个副本集实验节点在本阶段停止以释放虚拟机资源，但其数据目录和配置文件保留；任务六使用独立端口和独立数据目录，避免不同实验之间相互覆盖。

6.3.1 集群拓扑与部署边界

表 6-5 分片集群组件拓扑与端口配置

组件	副本集或角色	地址与端口	实验说明
Config Server	configRS	192.168.121.134:27219	单成员副本集
Shard 1	shard1RS	192.168.121.134:27211	单成员副本集
Shard 2	shard2RS	192.168.121.134:27212	单成员副本集
Router	mongos	192.168.121.134:27200	统一访问入口

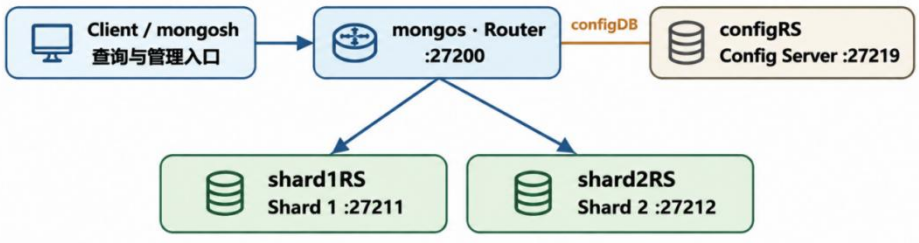


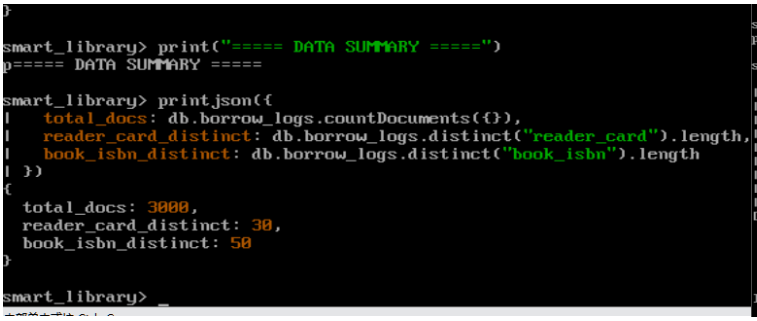
图 6-4 分片集群拓扑与端口规划

本实验将 Config Server 和两个 Shard 均配置为单成员副本集，主要考虑课程虚拟机的资源约束。该拓扑足以验证分片键、mongos 路由、Chunk 管理和数据跨 Shard 分布，但不具备生产环境所需的配置服务器冗余、跨故障域副本和整机故障恢复能力。

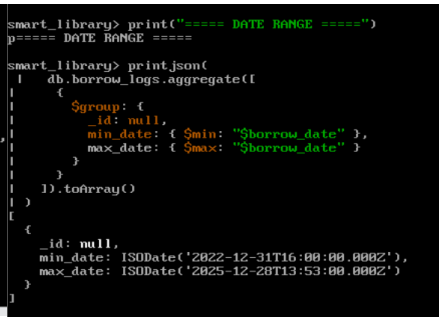
6.3.2 分片键选择与论证

borrow_logs 是平台中增长速度最快的业务集合，既要长期保存借阅流水，又要支持按读者、图书和时间进行查询与统计。因此，分片键不能只根据字段名称选择，而需要同时考察字段基数、现有数据分布、常用查询方式和持续写入时的热点风险。实验首先对 3000 条 borrow_logs 进行统计：reader_card 有 30 个不同值，每个值对应

100 条记录，最小值、最大值和平均值均为 100，标准差为 0；book_isbn 有 50 个不同值，每个值对应 60 条记录，最小值、最大值和平均值均为 60，标准差同样为 0。借阅时间的业务范围为 2023—2025 年，终端中的最早时间显示为 2022-12-31T16:00:00.000Z，这是 UTC 表示，对应北京时间 2023-01-01 00:00，不能据此将业务数据判断为从 2022 年开始。统计结果见图 6-2。



(a) 数据规模与候选字段基数



(b) 借阅时间范围



(c)reader_card 频率分布统计



(d) book_isbn 频率分布统计

图 6-5 borrow_logs 分片键候选字段统计分析

从当前实验数据看，reader_card 和 book_isbn 都没有明显倾斜，且 book_isbn 的基数实际上更高，因此不能把 reader_card 描述为比 book_isbn 更均匀。最终选择{reader_card: "hashed"}，主要依据是业务访问方式而非单一的均匀性指标：读者借阅历史、当前借阅情况和个人借阅记录通常以借阅证号作为完整查询条件，能够自然地形成定向路由场景；同时，reader_card 的哈希分片将不同读者键值映射到哈希空间，降低键值有序增长造成写入集中风险。对于 30 个读者、每个读者 100 条记录的当前数据集，该选择也能够为两个 Shard 的数据分布验证提供足够的键空间。

book_isbn 是本实验中可行的备选方案。它的基数为 50，高于 reader_card，且当前每个 ISBN 对应 60 条记录，同样不存在数据倾斜。未采用 book_isbn，并不是因为其分布不均，而是因为本项目将 borrow_logs 首先视为读者行为流水，按借阅证号查询一组历史记录具有更直接的业务含义；按图书维度的排行和统计通常还要经过聚合处理，单个 ISBN 等值条件不能覆盖全部分析需求。borrow_date 也可以支持时间范围查询，但普通范围分片会使新写入持续进入最新时间区间，长期运行时可能形成末端写入集中。因此，最终采用 hashed reader_card，在业务查询可路由性与写入分散之间取得课程实验范围内的平衡。

6.3.3 数据导入与实际分布

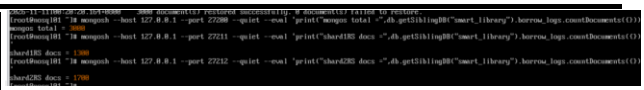
分片实验开始前，从任务三的 smart_library.borrow_logs 完整导出 3000 条记录，并通过 mongos:27200 恢复到已启用 {reader_card:"hashed"} 的 smart_library.borrow_logs 集合。mongorestore 输出显示 3000 documents restored successfully，0 failures。随后通过 mongos 查询总量，并分别连接两个 Shard 统计本地文档数。

表 6-6 borrow_logs 集合分片文档分布统计

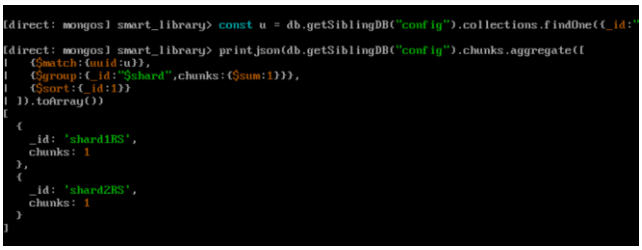
验证项目	实测结果
源集合与恢复结果	3000 条；mongorestore 成功 3000，失败 0
shard1RS 本地文档数	1300
shard2RS 本地文档数	1700
总和	1300 + 1700 = 3000
分布比例	约 43.3% / 56.7%，不是严格对半



（a）Shard 列表与 Balancer 状态



（b）真实文档发布



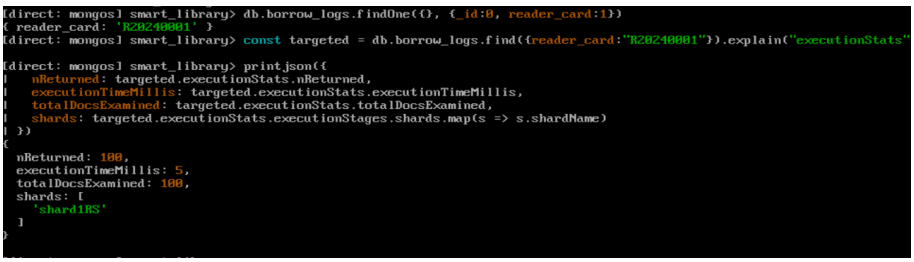
（c）Chunk 分布

图 6-6 分片集群最终状态与 borrow_logs 实际分布

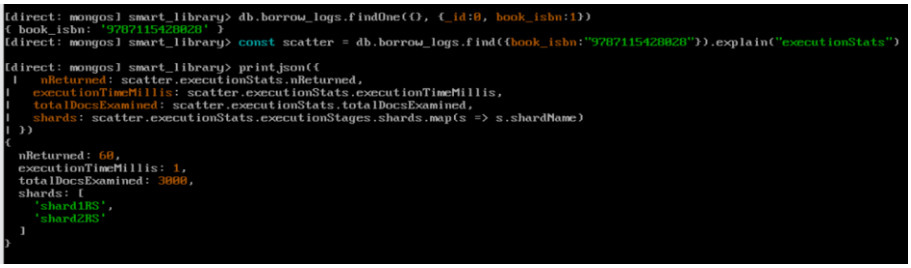
初始 Chunk 查询显示 shard1RS 和 shard2RS 各有 1 个 Chunk；最终文档统计显示两个 Shard 分别保存 1300 条和 1700 条记录。该结果证明真实业务数据已经由 mongos 路由并存储在两个 Shard，而不是只完成 Shard 注册。分布比例约为 43.3% 和 56.7%，因此只能表述为两个 Shard 均承担了业务数据存储，不能写成完全均匀分布。

6.3.4 查询路由验证

为验证分片键对 mongos 路由的实际作用，实验对比了一条包含完整 reader_card 分片键的定向查询和一条不包含分片键的 book_isbn 查询。定向查询使用 reader_card="R20240001"，返回 100 条记录，totalDocsExamined 为 100，只访问 shard1RS，executionTimeMillis 为 5；跨分片查询使用 book_isbn="9787115428028"，返回 60 条记录，totalDocsExamined 为 3000，同时访问 shard1RS 和 shard2RS，executionTimeMillis 为 1。



（a）基于 reader_card 分片键的定向查询



（b）基于非分片键 book_isbn 的查询，访问 shard1RS 与 shard2RS

图 6-7 定向查询与跨分片查询的路由行为对比

两类查询的主要差异体现在访问范围：完整分片键使 mongos 能够将请求定向到一个 Shard，而不包含分片键的查询需要访问两个 Shard 并汇总结果。由于实验数据量只有 3000 条，单次毫秒级耗时受运行环境和缓存影响较大，不能依据 5 ms 与 1 ms 的差异判断哪种查询性能更好；本实验以访问 Shard 数量和扫描文档范围作为主要验证依据。

6.3.5 访问控制、排错与最终状态

分片数据分布和查询路由验证完成后，集群统一启用访问控制，以减少部署阶段的认证变量。Config Server、两个 Shard 与 mongos 使用同一 keyfile 建立内部信任关系，并创建 shardingAdmin 管理账号。未认证执行 listShards 返回 Command listShards requires authentication；管理员认证后能够正常访问 shard1RS 和 shard2RS。认证重启后，27200、27211、27212 和 27219 均恢复监听，Balancer 最终状态为 true，borrow_logs. countDocuments({}) 仍为 3000。相关认证过程见附录。

认证重启期间曾出现 shard2RS 新进程 child process failed, exited with 1。排查发现旧 PID 4845 仍占用 27212；正常关闭旧实例后重新启动，新 PID 6650 成功监听。该问题说明多实例实验中，残留进程和端口占用会直接影响重启结果，处理时需要先核对进程状态，再执行启动操作。

综上，任务六达到任务书要求的核心结论：集群拓扑可用，两个 Shard 已注册并正常运行；smart_library. borrow_logs 的 3000 条真实数据已分布在 shard1RS 和 shard2RS；{reader_card:"hashed"} 能支持基于完整 reader_card 的定向路由，而不带分片键的 book_isbn 查询会访问两个 Shard；访问控制已启用且认证结果正确。由于 Config Server 和两个 Shard 均为单成员副本集并部署在同一台虚拟机，本结论限定于课程实验中的分片、路由和数据分布验证。

第 7 章 问题与总结

7.1 关键技术难点及排查过程

数据库部署、访问控制、分片集群运行以及性能验证过程中，项目出现了若干具有代表性的技术问题。相关问题均结合实际运行状态进行定位，并通过配置检查、端口与进程核对以及执行结果对比完成排查和验证。

单机 MongoDB 部署完成后，虚拟机内部访问正常，但 Windows 端 Compass 和 Python 无法连接 27018。检查 mongod 进程、监听端口和 bindIp 后均未发现异常，继续从宿主机测试目标端口，最终确认是 firewalld 未放行新增端口。补充 27018/tcp 规则后客户端恢复连接。该问题说明，远程访问异常需要同时检查数据库服务、监听配置和主机网络策略，而不能仅依据进程状态判断服务是否可用。

副本集启用 keyfile 认证时，文件所有者和权限已经按要求设置，但修改 SELinux 标签仍返回 Invalid argument。继续检查当前系统的安全上下文和策略后，发现参考配置并不适用于本机环境，因此没有继续强行修改标签，而是保留正确的文件权限，并通过管理员认证、未认证访问拒绝和节点正常通信来确认配置是否有效。后续认证能够正常工作，说明相关安全配置需要结合实际系统环境进行判断。

分片集群启用认证后重新启动 shard2RS 时，进程返回 child process failed, exited with 1。检查 27212 端口后发现旧 PID 4845 仍在运行，新实例因此无法绑定端口。正常停止旧进程并确认端口释放后重新启动，新 PID 6650 成功监听，集群随后恢复正常。该问题表明，在同一主机运行多个 MongoDB 实例时，服务启动异常还需要结合端口占用和实际进程状态进行排查。

索引优化实验中，最初通过`executionStats`获取的两组查询耗时均显示为 0ms，无法直接比较索引建立前后的实际耗时。为进一步验证优化效果，在保持查询条件一致的情况下，采用预热 100 次、正式执行 2000 次并重复测试 5 轮的方式补充计时。测试结果显示，图书类别与价格查询的平均耗时由 0.9883ms 降至 0.9475ms，变化较小；读者借阅历史查询由 2.3050ms 降至 1.5946ms，下降约 30.82%。结合执行计划及文档扫描数量的变化，进一步验证了复合索引对查询效率的改善作用。

7.2 项目总结与反思

本项目围绕智慧图书馆的实际业务需求完成了从数据库选型、数据结构设计到功能实现和分布式部署的完整验证。前期选型和建模确定了 MongoDB 在嵌套字段、可选属性、借阅流水和数字资源管理方面的适用性，并据此完成 books、readers、borrow_logs、audit_logs 等集合及索引设计。后续单机实现和 Python 应用验证了数据导入、检索统计、权限控制和 GridFS 文件管理等功能，说明前期设计能够落实到实际业务场景中。

在此基础上，副本集和分片集群分别验证了系统在高可用和水平扩展方面的能力。副本集完成了主节点故障后的自动选举和客户端恢复，分片集群则使 3000 条 borrow_logs 实际分布到两个 Shard，并验证了定向查询与跨分片查询的路由差异。最终使用 Compass 对数据总量和借阅状态聚合结果进行独立核对，结果与 mongosh 一致，并将统计结果导出作为交付文件。

整个实施过程表明，系统结论不能只依赖单一命令或单次耗时，而应结合数据量、执行计划、节点状态和实际查询结果综合判断。当前实验环境已完成课程要求范围内的核心功能验证、进程级故障转移和分片路由验证，但受单机虚拟机资源条件限制，实验结果尚不能代表跨主机生产环境下的高可用与扩展能力。